LAB NO. 09

STATIC DATA MEMBER AND METHODS

Lab Objectives

Following are the lab objectives:

1. Learn to define static data members in classes
2. Learn to define static member methods in classes

Student ID	**9864**	**Student Name**	**Ali Raza**
	Marks	**Obtained Marks**	**Comments**
Task 1	**20**		
Task 2	**20**		
Task 3	**20**		
Total Marks	**60**		

Lab Instructor

Lab Objectives and CLOs Mapping

Lab Objectives	CLOs		
	a	b	c
1			
2			
3			

Instructions

- This is individual Lab work/task.
- Complete this lab work within lab timing.
- Discussion with peers is not allowed.
- You can consult any book, notes & Internet.
- Copy paste from Internet will give you **negative marks**.
- Lab work is divided into small tasks, complete all tasks sequentially.
- Show solution of each lab task to your Lab Instructor.
- In-Lab Exercises/Tasks
- **Write your code at provided space after each question**
- **You need to upload code for all tasks at Google Class.**

C++ Static Data Members

Static data members are class members that are declared using **static** keywords. A static member has certain special characteristics which are as follows:

- Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
- It is initialized before any object of this class is created, even before the main starts outside the class itself.
- It is visible can be controlled with the class access specifiers.
- Its lifetime is the entire program.

Syntax

```
className {
     static data_type data_member_name;
     .....
}
```


```
className {
     static data_type data_member_name;
     .....
}
```


Static data members are useful for maintaining data shared among all instances of a class. The C++ Course explains how to implement static data members, ensuring you understand their significance in C++ programming.

Example

Below is the C++ program to demonstrate the working of static data members:

```cpp
// C++ Program to demonstrate the use of
// static data members
#include <iostream>
using namespace std;

// class definition
class A {
public:
    // static data member here
    static int x;
    A() { cout << "A's constructor called " << endl; }
};

// we cannot initialize the static data member inside the
// class due to class rules and the fact that we cannot
// assign it a value using constructor
int A::x = 2;

// Driver code
int main()
{
    // accessing the static data member using scope
    // resultion operator
    cout << "Accessing static data member: " << A::x
         << endl;

    return 0;
}
```

Output

```
Accessing static data member: 2
```

Defining Static Data Member

As told earlier, the static members are only declared in the class declaration. If we try to access the static data member without an explicit definition, the compiler will give an error.
To access the static data member of any class we have to define it first and static data members are defined outside the class definition. The only exception to this are static const data members of integral type which can be initialized in the class declaration.

Syntax

```
datatype class_name::var_name = value...;
```

For example, in the above program, we have initialized the static data member using the following statement:

```
int A::x = 10
```

Note: *The static data members are initialized at compile time so the definition of static members should be present before the compilation of the program*

Inline Definition of Static Data Member since C++ 17

The C++ 17 introduced the inline definition of the static data members of type integral or enumeration which was not allowed in the previous standards. This simplifies the definition of the static data members.

Syntax

```
// inside class definition
...
static inline data_type var_name = value/expression;
....
```

Accessing a Static Member

We can access the static data member without creating the instance of the class. Just remember that we need to initialize it beforehand. There are 2 ways of accessing static data members:

1. Accessing static data member using Class Name and Scope Resolution Operator

The class name and the scope resolution operator can be used to access the static data member even when there are no instances/objects of the class present in the scope.

Syntax

```
Class_Name :: var_name
```

Example

```
A::x
```

2. Accessing static data member through Objects

We can also access the static data member using the objects of the class using dot operator.

Syntax

```
object_name.var_name
```

Example

```
obj.x
```

> ***Note:*** *The access to the static data member can be controlled by the class access modifiers.*

Example to Verify the Properties of the Static Data Members

The below example verifies the properties of the static data member that are told above:

```cpp
// C++ Program to demonstrate
// the working of static data member
#include <iostream>
using namespace std;

// creating a dummy class to define the static data member
// it will inform when its type of the object will be
// created
class stMember {
public:
    int val;
    // constructor to inform when the instance is created
    stMember(int v = 10): val(v) {
        cout << "Static Object Created" << endl;
    }
};
```

```
// creating a demo class with static data member of type
// stMember
class A {
public:
    // static data member
    static stMember s;
    A() { cout << "A's Constructor Called " << endl; }
};

stMember A::s = stMember(11);

// Driver code
int main()
{

    // Statement 1: accessing static member without creating
    // the object
    cout << "accessing static member without creating the "
            "object: ";
    // this verifies the independency of the static data
    // member from the instances
    cout << A::s.val << endl;
    cout << endl;

    // Statement 2: Creating a single object to verify if
    // the seperate instance will be created for each object
    cout << "Creating object now: ";
    A obj1;
    cout << endl;
```

```
    // Statement 3: Creating multiple objects to verify that
    // each object will refer the same static member
    cout << "Creating object now: ";
    A obj2;
    cout << "Printing values from each object and classname"
         << endl;

    cout << "obj1.s.val: " << obj1.s.val << endl;
    cout << "obj2.s.val: " << obj2.s.val << endl;
    cout << "A::s.val: " << A::s.val << endl;

    return 0;
}
```

Output

```
Static Object Created
accessing static member without creating the object: 11

Creating object now: A's Constructor Called

Creating object now: A's Constructor Called
Printing values from each object and classname
obj1.s.val: 11
obj2.s.val: 11
A::s.val: 11
```

Note: *In C++, we cannot declare static data members in local classes.*

LAB TASKS

Task 1 (15 mark)

Create a **SavingsAccount** class. Use a static data member **annualInterestRate** to store the annual interest rate for each of the savers. Each member of the class contains a private data member **savingsBalance** indicating the amount the saver currently has on deposit. Provide member function **calculateMonthlyInterest** that calculates the monthly interest by multiplying the balance by **annualInterestRate** divided by 12; this interest should be added to **savingsBalance**.

Provide a static member function **modifyInterestRate** that sets the static **annualInterestRate** to a new value.

Write a main function to test SavingsAccount class. Instantiate two different objects of class SavingsAccount, saver1 and saver2, with balances of $2000 and $3000, respectively. Set the annualInterestRate to 3 percent. Then calculate the monthly interest and print the new balances for each of the savers.
Then set the annualInterestRate to 4 percent, calculate the next month's interest and print the new balances for each of the savers.

Program:

```
#include <iostream>

using namespace std;


class SavingsAccount {

private:
```

```
double savingsBalance;
static double annualInterestRate;

public:
    SavingsAccount(double balance) {
        savingsBalance = balance;
    }
    void calculateMonthlyInterest() {
    double interest = (savingsBalance * annualInterestRate) / 12;
    savingsBalance += interest;
    }

void displayBalance() const {
    cout << "Balance of Acc: " << savingsBalance << endl;
    }

  static void modifyInterestRate(double newRate) {
annualInterestRate = newRate;
```

```
    }
};
double SavingsAccount::annualInterestRate = 0;

int main() {
    SavingsAccount saver1(20000.0);
    SavingsAccount saver2(30000.0);
    SavingsAccount::modifyInterestRate(0.03);

    cout << "Balanc after 1 month at 3% annual interest:" << endl;
    saver1.calculateMonthlyInterest();
    saver2.calculateMonthlyInterest();
    saver1.displayBalance();
    saver2.displayBalance();
    SavingsAccount::modifyInterestRate(0.04);

    cout << "Balance after next month at 4% interest:" << endl;
    saver1.calculateMonthlyInterest();
```

```
    saver2.calculateMonthlyInterest();

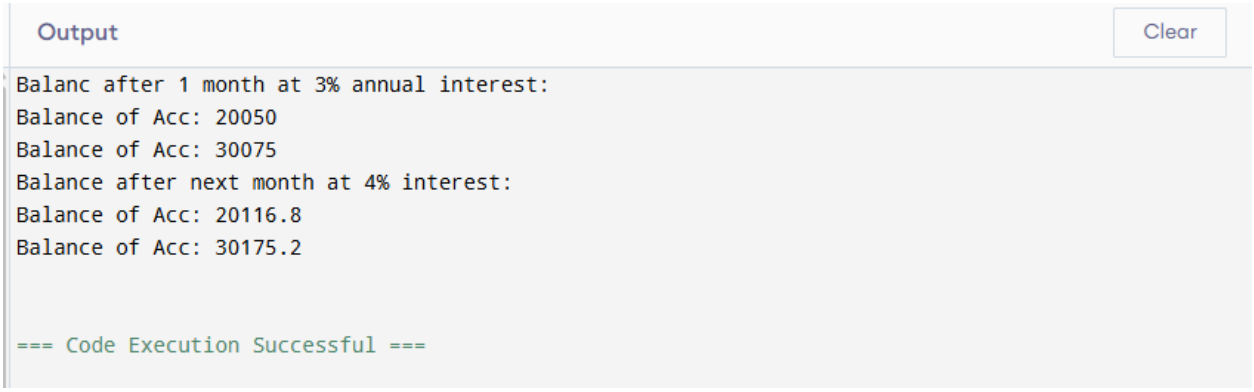
    saver1.displayBalance();

    saver2.displayBalance();


    return 0;

}
```

Output:



Task 2 **(15 mark)**

Write program to count the number of objects created and destroyed for a class using static data members and static member functions. You can develop any class of your choice.

Program:

```
#include <iostream>
using namespace std;
class ali{
private:
   static int createCount;
public:
   ali() {
     createCount++;
       cout << "Object created:" << createCount << endl;
   }
   ~ali() {
       createCount--;
     cout << "object destroy:" << createCount << endl;
   }
  static void displayCounts() {
        cout << " Before create and destroyed " << createCount << endl;

   }
};
int ali::createCount = 0;

int main() {
   ali::displayCounts();
   ali a1, a2;
   ali::displayCounts();
   return 0;
```

}

Output:



Task 3 (15 mark)

Create a **Calculator** class that has following methods:

1. sum,
2. multiply
3. divide
4. modulus
5. sin
6. cos
7. tan

The user should be able to call these methods without creating an object of Calculator class. Demonstrate working of your class.

Program:

```
#include <iostream>
using namespace std;

class calculator {
public:
    static double sum(double a, double b) {
        return a + b;
    }

     static double multiply(double a, double b) {
        return a * b;
    }

    static double divide(double a, double b) {
        if (b == 0) {
            cout << "Error: Division by zero!" << endl;
            return 0;
        }
        return a / b;
    }

    static int modulus(int a, int b) {
        if (b == 0) {
            cout << "Error: Modulus by zero!" << endl;
            return 0;
```

```
    }
    return a % b;
}

static double power(double base, int exp) {
    double result = 1;
    for (int i = 0; i < exp; i++) {
        result *= base;
    }
    return result;
}

static long long factorial(int n) {
    long long result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}

static double sin(double x) {
    double result = 0;
    for (int n = 0; n < 10; n++) {
        double term = power(-1, n) * power(x, 2*n+1) / factorial(2*n+1);
        result += term;
```

```
        }
        return result;
    }

    static double cos(double x) {
        double result = 0;
        for (int n = 0; n < 10; n++) {
            double term = power(-1, n) * power(x, 2*n) / factorial(2*n);
            result += term;
        }
        return result;
    }

     static double tan(double x) {
        double c = cos(x);
        if (c == 0) {
            cout << "Error: tan undefined " << endl;
            return 0;
        }
        return sin(x) / c;
    }
};

int main() {
    cout << "Sum: " << calculator::sum(10, 5) << endl;
```

```
    cout << "Multiply: " << calculator::multiply(10, 5) << endl;
    cout << "Divide: " << calculator::divide(10, 5) << endl;
    cout << "Modulus: " << calculator::modulus(10, 3) << endl;

    double angle = 1.0;
    cout << "Sin: " << calculator::sin(angle) << endl;
    cout << "Cos: " << calculator::cos(angle) << endl;
    cout << "Tan: " << calculator::tan(angle) << endl;

    return 0;
}
```

Output:

